

Apple 10-Q Multimodal RAG

Final Design, Methodology & Engineering Report

Assessment: RAG-based Information Retrieval System over Apple Inc. Q3 2022 Form 10-Q
Scope: Text • Tables • Figures
Implementation: Python • FastAPI • BM25 • FAISS • Sentence Transformers • Cross-Encoder • OpenAI • Docker

2. Assignment Requirements

Requirement	Implementation
Text retrieval	PyMuPDF page extraction + semantic text chunks
Table retrieval	pdfplumber extraction + semantic table reconstruction
Figure retrieval	Page/figure rendering + separate figure modality
RAG generation	Evidence-constrained OpenAI generation
Citations	Page/modality citations with validation against retrieved evidence
Unsupported queries	Explicit abstention behavior
Maintainability	Typed models, modular core, tests, configuration, Docker
Submission artifacts	GitHub repository + separate methodology/design PDF

3. System Architecture

Source PDF → Ingestion → Multimodal chunks → BM25 + FAISS → Hybrid candidate pool → Cross-encoder reranking → Table-aware scoring → Evidence selection → Grounded generation → Citation validation → FastAPI response.

The architecture separates concerns into ingestion, retrieval, generation, pipeline orchestration, API delivery, and evaluation. This allows changes to chunking, retrieval, or prompting without coupling them to the HTTP layer.

4. PDF Ingestion Pipeline

4.1 Text extraction

PyMuPDF is used for page-level text extraction. Page metadata is retained so evidence can be traced back to the source document. Text chunks retain document ID, page, modality, source, and optional bounding-box/image metadata.

4.2 Table extraction and reconstruction

Challenge: A PDF table is primarily a visual layout. Naive extraction can separate column headers from values and lose the relationship between a financial metric, reporting period, and number. This is particularly dangerous for financial QA because a correct number paired with the wrong period can produce a plausible but incorrect answer.

Solution: pdfplumber is used to identify tables, followed by cell cleaning and semantic reconstruction. Tables are represented with explicit column definitions and row-level period-to-value mappings. For example, the income statement records Total net sales as 82,959, 81,434, 304,182, and 282,457 under their corresponding reporting periods.

4.3 Figure handling

Figures are treated as a separate modality. Relevant pages are rendered and the image path is retained in metadata. The architecture supports optional vision-derived descriptions so information that is not represented in the PDF text layer can still participate in retrieval and grounded generation.

4.4 Final ingestion result

Modality	Final chunks
Text	28
Table	31
Figure	1
Total	60

5. Retrieval Architecture

5.1 Modality-aware retrieval

Retrieval is performed independently across text, table, and figure modalities. This prevents the large volume of narrative text from completely dominating table and figure evidence.

5.2 Hybrid retrieval

BM25 supplies lexical matching for exact financial terms, dates, and entities. Sentence-transformer embeddings and FAISS supply semantic retrieval for paraphrased questions. The candidate sets are combined before reranking.

5.3 Cross-encoder reranking

Candidate question/chunk pairs are scored with a cross-encoder. The scores are normalized before being combined with structured table relevance.

5.4 Table-aware ranking

Evaluation exposed a specific failure mode: a generic cross-encoder could rank a semantically related narrative paragraph above the exact financial table. For table chunks, the final score therefore uses 85% normalized cross-encoder relevance and 15% structured table relevance. The structured signal considers metric, period, date, direct-value, and lexical matches.

5.5 Evidence diversity

Final evidence selection retains high-quality table evidence where necessary and avoids allowing near-duplicate candidates to consume the entire context window.

6. Grounded Generation, Citations and Abstention

The generation layer receives retrieved evidence and is instructed to answer only from that evidence. Numeric questions prefer direct table evidence. Unsupported facts are not invented. Citation syntax is parsed and validated against the page/modality pairs actually present in retrieved evidence.

The system also supports abstention. If the document does not contain enough evidence, the model is instructed to say so. This is explicitly evaluated with an out-of-scope question.

7. API and Deployment

GET /health — liveness endpoint.

GET /ready — readiness endpoint confirming that the RAG pipeline has loaded.

POST /query — accepts a question and returns the grounded answer, citations, latency, and retrieved evidence.

Docker/Compose packages the API and runtime dependencies. Configuration is provided through environment variables; API secrets are kept outside Git.

8. Evaluation Methodology

The final evaluation suite contains 10 questions covering text retrieval, financial tables, geographic revenue data, a numerical calculation, figure understanding, and unsupported-question abstention. Evidence is evaluated using acceptable page/modality alternatives rather than requiring a single exact chunk.

Metric	Final result
Questions	10
Recall@1	60.00%
Recall@5	100.00%
Recall@8	100.00%
MRR	0.7750
Answer accuracy	100.00%
Citation accuracy	100.00%
Abstention accuracy	100.00%
Mean latency	5724.55 ms
Median latency	5042.39 ms
Maximum latency	8178.92 ms

Recall@5 and Recall@8 are both 100%, indicating that relevant evidence is consistently recovered within the candidate set even when the first-ranked item is sometimes a related narrative chunk. Answer, citation, and abstention checks all pass on the final benchmark.

9. Benchmark Design Note: Percentage Calculation

For the nine-month total net sales change, the arithmetic calculation is approximately $(304,182 - 282,457) / 282,457 \times 100 = 7.69\%$. The source document reports the increase rounded to 8%. The evaluator therefore accepts 7.69%, 7.7%, or 8% to recognize both the calculated result and the document-reported value.

10. Challenges and Solutions

Challenge	Solution
PDF text is layout-oriented	Page-aware extraction with source metadata
Table column/value relationships can be lost	Explicit table reconstruction with period-to-value mappings
Repeated financial dates can be ambiguous	Explicit column definitions and row semantics
Generic reranking can prefer narrative text	Table-aware structured relevance signal
Exact terms may be missed by semantic retrieval	BM25 + dense retrieval
Figures contain information outside text	Separate figure modality and rendering
LLM may hallucinate	Evidence-only generation and abstention
Citations may be unsupported	Citation validation against retrieved evidence
Large source/index files should not enter Git	.gitignore and reproducible ingestion
Runtime differences	Docker/Compose

11. Assumptions and Limitations

The supplied Apple Q3 2022 10-Q is treated as the authoritative source. The evaluation set contains 10 questions and is therefore a small benchmark. The current mean latency is approximately 5.7 seconds in the tested environment. Complex tables with merged cells or unusual footnotes may require stronger cell-level extraction. Figure understanding depends on the available visual representation/description path. The current indexes are local rather than persistent

distributed stores.

12. Production Improvements

- OCR fallback for scanned/image-only PDFs.
- Cell-level coordinates and stronger merged-cell/footnote handling.
- Dedicated vision-language retrieval for complex figures.
- Persistent vector and lexical indexes.
- Query classification to adapt retrieval depth by modality.
- Embedding/reranking caching and batching.
- Larger regression and human-evaluation datasets.
- CI/CD evaluation gates.
- Authentication, rate limiting, tracing, and observability.
- Latency optimization through model warm-up and cached inference.

13. Project Structure and Reproducibility

The repository separates application code, ingestion/evaluation scripts, tests, artifacts, and configuration. The supplied PDF is placed under `data/` before ingestion; generated indexes are written under `artifacts/index/`. Secrets are loaded through `.env` and are not committed.

Local ingestion

```
python -m scripts.ingest
```

Local evaluation

```
python -m scripts.evaluate
```

Docker

```
docker compose up --build -d
```

14. Submission Contents

The GitHub repository contains the complete project code, tests, Docker configuration, evaluation results, and documentation. The separate methodology/design PDF is included under `artifacts/`. The repository intentionally excludes the supplied source PDF, generated index, and API credentials.

GitHub: <https://github.com/harshrajshs/apple-10q-multimodal-rag>

Design report: `artifacts/Apple_10Q_Multimodal_RAG_Design_Report.pdf`